

De-centràX Security Review

Conducted by: **Georgi Trachev**

Report Version 1.0

February 7, 2025

Contents

1. About Georgi Trachev

2. Disclaimer

3. About De-centràX

4. Risk Classification

4.1. Impact

4.2. Likelihood

4.3. Actions required by severity level

5. Security Assessment Summary

6. Executive Summary

7. Findings

1. About Georgi Trachev

Georgi Trachev is a smart contract security researcher. Throughout his experience he has uncovered numerous critical vulnerabilities in a wide variety of protocols. Reach out on X [here](#).

2. Disclaimer

Smart contract security reviews are a time, resource and expertise bound effort where security researchers do their utmost to find as many vulnerabilities as they can. They are not a 100% guarantee of the security of smart contracts.

3. About De-centràX

De-centràX is a decentralized protocol, deployed on the PulseChain. It is centred around the APED coin which has a tax structure and 2 liquidity rivers (balancer pools). The liquidity pools are initialized upon creation of the token and can be used by users to trade or provide liquidity. Token holders are also eligible for earning rewards which accumulate in the ReflectionDistributor smart contract.

Lastly, users can stake APER tokens to earn more APED yield through the Staking smart contract.

4. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
----------	--------------	----------------	-------------

Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4.1 Impact

High: Causes a significant loss of funds, the protocol is directly compromised

Medium: A minor amount of funds at loss, or a functionality of the protocol can be affected

Low: Unexpected behaviour in the protocol, but does not present a significant harm

4.2 Likelihood

High: Direct attack vector; cost of attack negligible, compared to funds at risk

Medium: A sophisticated attack vector that is incentivized only in some conditions but is still likely to occur

Low: Requires many unlikely assumptions or the attacker is not incentivized

4.3 Actions required by security level

High: Issue **MUST** be fixed

Medium: Issue **SHOULD** be fixed

Low: Issue **MAY** be fixed

5. Security Assessment Summary

Overview

Repository: <https://github.com/De-centraX/prc20-tax>

Commit hash: cf06e01fdd487a9ca7a74110451a565e88e15167

Scope

The smart contracts included in the scope of the security review:

- src/ReflectionDistributor.sol
- src/Staking.sol
- src/TokenOwner.sol
- src/tokens/Token.sol
- src/tokens/TokenP1.sol
- src/tokens/TokenP2.sol
- src/utils/SafeMath.sol

Timeline

Audit start: February 1, 2025

Preliminary report: February 6, 2025

6. Executive Summary

Throughout the security review, conducted in the period February 1-6, 2025, 8 issues were discovered.

Findings count

Medium Risk	6
Low Risk	2

7. Findings

7.1 Medium Findings.

[M1] PLS and ORD tokens are lost when handling the pool tax

Severity: *Medium*

Context: Token._sendToORDTreasury.sol#L265

Description: In the `_addToPLSLP` and `_addToORDLP` tokens will be lost due to the way liquidity is added. In `_addToPLSLP` half of the accumulated fees are swapped for PLS. The received PLS and the other half of the fees is then added as liquidity to the pool. The issue is that after the first swap the price of PLS in the pool will grow. As a result, not the entire `amountReceived` will be provided to the pool and a portion of it will be returned to the contract. Currently, the leftover PLS will be stuck.

The same issue occurs in the `_addToORDLP` function as well, however, there the issue will be more problematic as there more ORD tokens are received from the TokenOwner contract. Those tokens will also not be included when adding liquidity and will remain stuck in the contract.

Recommendation: Consider transferring any leftover PLS/ORD tokens to the Treasury.

Resolution: Resolved.

[M2] Incorrect APED tax allocation**Severity:** *Medium***Context:** Token._handlePoolTax.sol#L240

Description: The tax structure of the APED token intends to allocate 30% of token transfers in the first 2 hours of the launch for taxes. The issue is that due to the way taxes for the first 2 hours were calculated the taxes would have been only 25%. As a result, during the first 2 hours of launch less fees than expected would have been accrued.

Recommendation: Tax allocation percentages should be increased.

Resolution: Resolved.

[M3] Forming token liquidity pools can be bricked**Severity:** *Medium***Context:** Token.formPools.sol#L146

Description: The *formPools* function of Token.sol attempts to create liquidity pools for the APED token, paired with a variety of other tokens. The issue is that if a user has already created a pool with the same pair of tokens *factory.createPool* will revert. As a result, the forming of the liquidity pools can be bricked.

The same issue can be found in the TokenP1 and TokenP2 smart contracts which also attempt to create liquidity pools through *factory.createPool*.

Recommendation: The issue can be avoided by calling *factory.createPool* only if a pool for the specific tokens has not been created.

Resolution: Acknowledged.

[M4] The Staking contract will lose a large portion of its earnings**Severity:** *Medium*

Context: Token._apexToAperVault.sol#L317

Description: In the Token contract when the accrued transfer fees are distributed a part of them are used to increase the Staking contract's APED balance which is used for rewarding stakers. The issue is that the `_apexToAperVault` function implements the `_ignoreTax` modifier. As a result, the Staking contract will not be minted ReflectionDistributor shares, thus it will not be rewarded the yield that APED token holders are eligible to receive. As a result, the Staking contract will lose a major portion of its ReflectionDistributor yield.

Recommendation: One way of solving the issue is by calling `ordDistirbutor.setShare` inside of `_apexToAperVault` in order to notify ReflectionDistributor of the contract's balance increase.

Resolution: Resolved.

[M5] Incorrect whitelist validation

Severity: *Medium*

Context: Token._update.sol#L203

Description: The whitelist validation has been implemented incorrectly. In all token contracts when whitelisted users transfer or receive tokens the transfer should not incur any fees. The issue is that fees are not charged only if both the receiver and the sender of the tokens are whitelisted. As a result, when the Staking contract transfers reward tokens to stakers, fees would be taken out of the rewards, even though the Staking contract should be whitelisted.

The issue will also occur in the TokenP1 and TokenP2 contract as they also implement the same whitelist system.

Recommendation: Whitelist validation should tax users only if both the receiver and the sender are not whitelisted.

Resolution: Resolved.

[M6] Lack of slippage protection when swapping tokens through the exchange

Severity: *Medium*

Context: Token._sendToORDTreasury.sol#L265

Description: In the Token contract when the accrued transfer fees are distributed a part of them are swapped through a decentralized exchange. The issue is that when the swaps are performed there is a lack of slippage protection. As a result, there is a risk of sandwich attacks where the swapped amount will be less than expected.

Similarly, the issue can be found in the TokenP1 and TokenP2 contracts when distributing fees.

Recommendation: One way of solving the issue is by calculating the expected swap output amount off-chain and supplying it as an input when distributing fees.

Resolution: Acknowledged.

7.2 Low Findings

[L1] Low APED transfers may revert

Severity: *Low*

Context: Token._apexToAperVault.sol#L320

Description: When APED tokens are transferred a portion of the transfer fees are distributed to the APER Staking contract where they are used to calculate the current reward rate. The issue is that if the transfer amount is too low the fees delegated to the Staking contract will be so low that the reward rate calculation will round down to 0. As a result, the Staking contract will revert causing the transfer to fail.

Recommendation: Consider implementing a try/catch block inside of `_apexToAperVault` when the function calls `notifyRewardAmount`. That

way if the reward rate rounds down to 0 the transfer will not revert, instead the fees that should have been transferred to the Staking contract can be transferred to the Treasury.

Resolution: Acknowledged.

[L2] Staking rewards can be lost

Severity: *Low*

Context: Token._apexToAperVault.sol#L319

Description: In the Token contract a portion of the APED transfer fees are distributed to the APER Staking contract where they are used as rewards for stakers. The Staking contract's reward per share is calculated based on the current reward rate and the number of total shares, where if users are yet to stake into the contract and there are no total shares the reward per share remains at 0. However, even if there are no shares, rewards can still accumulate in the contract. The issue is that these rewards will not be later distributed to the stakers but will remain stuck in the Staking contract.

Recommendation: Consider transferring APED fees to the Staking contract only if the total shares are more than 0, otherwise they should be redirected to the Treasury.

Resolution: Resolved.